

The Cognitive Firm: A Multidivisional-Form and Audit-Institution Architecture for Governing Self-Evaluating AI

Abstract

AI systems now grade their own work. Organizational economics has studied this for sixty years, where AI safety calls it *alignment*. When one process produces and judges its output under optimization pressure, it acquires an adversarial gradient against its own evaluation signal, and the symptoms are specification gaming, metric inflation, and fabricated compliance. Chandler (1962) documented a structurally similar pathology in the single-form firm, which a general office cured by judging divisions on data they could not touch. We carry that cure inward: we separate generation from evaluation behind a layer neither controls, and we borrow the independence guarantees of financial audit. Together they form the **Audited Multidivisional Architecture** of three structural separations and three institutional guarantees, each paired with the failure its absence has produced, most visibly the collapse of auditor independence at Arthur Andersen. We turn these familiar parts inward against a self-evaluating system to predict where its governance fails. On one operated system, the failures that survive a strict deterministic floor are overwhelmingly judgment-laden, and what is left tracks the score while pressure leaves it unchanged, so we read this as descriptive: the checks do not relocate gaming. A second, conditional claim follows agency theory, where those checks earn their overhead only as the principal goes absent and the governed properties become mechanically checkable. Our evidence is one system, one operator, with failures tagged by model judges, so it proves the architecture buildable without showing that it generalizes. Recursive-AI self-improvement is a problem of firm design and audit institutions as much as of model capability.

Keywords: AI governance · AI ethics · self-evaluating AI · specification gaming · independent audit · agency theory

1 Introduction

Systems that run improvement loops, generating a candidate, scoring the candidate, and conditioning the next candidate on that score, are now common. What remains absent is a settled account of *why* such loops degrade under optimization pressure, and of *what structural response* prevents the degradation. The most common framings are behavioural. Train the model to hold the right values, as in Constitutional AI, or shape the rewards, as in RLHF, while adjacent work on scalable oversight, debate, evaluations, and process supervision varies the signal or the procedure. All of this operates on what the model *says* and how the output is scored, and leaves open a question about what the system can *structurally do*: when the same optimizing process that produces an output also certifies that output, what stops the certification from being gamed?

Organizational economics has studied co-located generation and evaluation since Chandler’s (1962) account of the firm whose revenue divisions reported on their own performance, with the self-serving accounting and strategic drift later read as managerial opportunism (Williamson, 1975), corrected only by a structural change: a general office that monitored divisions using objective data

under the office’s control, separated from the divisions under judgment. The control problem has the same shape in recursive AI. That field’s institutional solutions, the *multidivisional form* and, beneath the form, the *independent audit profession*, supply a design vocabulary for governing self-evaluating systems.

Modeling AI oversight on an older verification institution is well established. A growing literature models *AI auditing* on *financial auditing* (Mökander and Floridi 2021; Mökander et al. 2024; Raji et al. 2020, 2022), borrowing the profession’s methodologies, attestations, and independence requirements, and borrowing its scandals too, so that Enron and the collapse of Arthur Andersen recur as the cautionary spine of that work. We extend the same precedent one layer inward. Where AI-auditing scholarship asks how an *external* party should verify a finished model, we ask how the *internal* architecture of a self-evaluating system should be organized so that verification becomes structurally possible at all. Applied to the firm’s own form, the two institutions answer this inward question. Our nearest technical neighbour is the AI-control literature on trusted and untrusted protocols (Greenblatt et al. 2023), which places a non-gameable component where it can check a more capable optimizer; the organizational reading then specifies why that component sits outside the loop, what makes it trustworthy once installed, and where the gaming it does not stop reappears.

Pushed for literal similarity, the analogy between a 1920s industrial enterprise and a language-model loop will mislead. Following recent work on the role of analogy in AI governance (Hatz 2025), we take from it what cross-domain analogies legitimately supply: a conceptual vocabulary and the cautionary lessons of the source domain’s failures, never a policy transplant that would require the two domains to be alike. We adopt the multidivisional form’s *separation* logic and the audit profession’s *independence* logic, and we decline its claims about divisional autonomy, market allocation, and human organizational culture (Section 6).

Our architecture (Section 3) sets three structural separations and three institutional guarantees, each mapped to a concrete recursive-AI primitive and each paired with the historical failure that its absence produced. From it we draw two predictions. The first is observable on a running system: a hard layer of model-free checks does not remove the adversarial gradient, so the residual lands on the surface a rule cannot govern, the judgment a fixed check cannot make. The second is conditional, in the absentee-owner sense of agency theory: that overhead is justified only as the principal becomes absent and the governed properties become deterministically checkable, and is pure drag otherwise. We illustrate both on one operated system (Section 5), whose observations are consistent with each prediction, and we close on the institutional endpoint the audit analogy implies, where the auditor-independence and liability guarantees that recursive AI still lacks would have to be built.

Self-evaluating AI is a problem of firm design and audit institutions as much as of model capability. The security engineer’s trusted monitor outside the loop, the agency-cost conditional on when monitoring pays, the audit profession’s trustworthiness guarantees, and the displacement of pressure under defense-in-depth each address part of the problem; once a system both produces and grades its own work, they cohere into six primitives. On one operated system, the failures that survive a strict deterministic floor are overwhelmingly judgment-laden: across 2,780 governed iterations they run 103 to 5 where the checks were actively engaged. What is left lands where a rule cannot reach. We read this as descriptive, since the checks do not drive gaming there.

2 Definitions and Scope

A few terms recur throughout, and the classification criterion carries the most weight.

An AI system is **self-evaluating** (or recursive) when the output of one iteration is scored and that score conditions the next iteration, by processes the system controls. In the paradigm case an

agent loop proposes a candidate, runs a check the loop also generated or selected, and uses the result to decide what to try next. **Generation** produces a candidate and **evaluation** certifies or scores it. Generation and evaluation are *co-located* when one optimizing process performs both, so that the parameters or prompts producing the output also determine whether the output passes. Under sustained optimization pressure this co-location becomes the pathological condition at issue: the optimizer’s gradient points toward satisfying the evaluation, and any gap between the letter of the evaluation and the controlling intent becomes a basin of attraction.

A system is **unitary** (by analogy, U-form) when the optimizing process produces or can soften the governing evaluation, as when an LLM both writes code and judges that code, or when a multi-agent loop’s coordinator is a language model the optimized agents can persuade. A system is **audited multidivisional** (M-form) when at least one governing constraint is **deterministic** and **cannot be softened by model output**. We set the bar deliberately low: a single rule-bound, fail-closed enforcement constraint qualifies a system however probabilistic the rest of the pipeline remains. Read structurally, the criterion asks whether any part of the evaluation lies *outside the optimizer’s reach* whatever the system’s surface behaviour.

The **principal** is the accountable party on whose behalf the system runs and who authors the constraints, an analog of the firm’s owner and, later, of the audit’s client. The **enforcement floor** is the set of deterministic, model-free constraints the principal installs outside the optimizing loop. We call it a *floor* because it is the minimum the optimizer cannot soften or argue past, the hard bottom beneath everything the model controls, and it is the object the rest of the paper is about. Determinism does not buy infallibility, since a constraint can fail-close reliably and still test the wrong property (Section 5.3), so the floor’s *content* and *independence* stay first-class problems (Section 3.4–Section 3.5) that determinism does not settle on its own.

Our evidence comes from **one operated system run by a single operator** (Section 5), which shows the framework is buildable and yields observations consistent with the predictions without establishing general applicability. Independent replication, operation by multiple operators, and a controlled efficiency comparison fall out of scope for future work (Section 8 and Section 9).

3 The Audited Multidivisional Architecture

Three structural separations come from the multidivisional form and three institutional guarantees from the audit profession. The cross-domain transfer they rest on has to be justified first.

Importing an older verification institution into AI governance has ample precedent. Schultz and Seele (2022) transfer the apparatus of business ethics, Hatz (2025) reviews the nuclear analogy, and Mökander and colleagues (2021, 2023, 2024) along with Raji and colleagues (2020, 2022) model AI auditing on financial auditing. We import a *specific structural solution* from corporate governance into the internal design of a recursive system: the separation of generation from evaluation institutionalized as independent audit. Where Chandler’s single-form firm concentrated operational execution and performance evaluation under one roof and produced self-serving reporting (Williamson, 1975), a recursive AI system concentrates generation and evaluation in one optimizing process and produces specification gaming, and in both the absence of a structural firewall creates an adversarial gradient against the evaluation signal. The parallel breaks down too. Firms are legal persons with intentional states while AI systems are not, and corporate restructuring unfolds over years while model loops iterate in minutes. These disanalogies are real, yet they bear on the source domain’s sociology and leave untouched the borrowed control structure, and where they bite they tend to sharpen the case, since faster iteration makes a deterministic floor more necessary. We adopt the multidivisional form’s separation logic and the audit profession’s independence logic, declining the analogy’s claims about

full divisional autonomy, market allocation, and organizational culture (developed in Section 6).

As a framework, the six primitives divide into three structural and three institutional, each carrying a source-domain homolog, a concrete realization in a recursive-AI system, and the failure that absence has historically produced (Table 1).

Table 1: The Audited Multidivisional Architecture.

	Primitive	Source-domain homolog	ho-	Recursive-AI realization	Failure its absence produced
<i>Structural: from the multidivisional form</i>					
S1	Separation of generation from evaluation	general office operating division (Chandler 1962)	vs.	the evaluator is not the optimizer that produced the output	self-serving divisional reporting
S2	A deterministic enforcement floor as integration primitive	a deliberate integration mechanism for differentiated units (Lawrence & Lorsch 1967)		model-free checks: stage order, scope/drift, fail-closed stops	negotiated, game-able evaluation
S3	An exogenous resource clock for closure	capital-allocation / quarterly review		a stop condition external to the optimizing loop	runaway loops; paralysis without closure
<i>Institutional: from the audit profession</i>					
I1	Rule-boundedness	GAAP / published methodology		a public, versioned rule-specification library	rules set by the audited party
I2	Independence	audit/advisory firewall		verifier outside the loop; cross-family separation	Arthur Andersen / Enron (2001)
I3	Attestation (liability open)	qualified opinion + partner signature		principal-signed, machine-verifiable run attestation	unverifiable / advisory-only sign-off

3.1 Separating Generation from Evaluation

We take the first primitive from Chandler: the unit that produces output must not be the unit that certifies the output. In the multidivisional form, strategic oversight sat in a general office that monitored divisions using objective data, structurally separated from the divisions under performance judgment. In a recursive system the evaluation governing a loop must be neither produced nor softenable by the optimizer that generated the candidate. Absent that separation, self-serving reporting follows: in the firm, inflated divisional results; in the loop, a candidate that satisfies a self-authored check.

The multidivisional form is defined by separation together with *bounded divisional autonomy*, where divisions hold operating authority inside a charter set above them, and a recursive system instantiates this only partly. The system’s components get **bounded operational autonomy within a principal-signed charter**, so the generative agent decides *what to try next* within fences it cannot move, while strategic authority and the monitoring data stay with the enforcement layer. Separation under a charter with objective monitoring makes the homology structural, and thinner than a Chandler division’s strategic independence.

3.2 The Deterministic Enforcement Floor

Separation creates a coordination problem: differentiated units have to be re-integrated without collapsing back into co-location. Lawrence and Lorsch (1967) showed that the more differentiated an organization, the more deliberate its integration mechanisms must be. We use the deterministic enforcement floor as that integration mechanism, coordinating the differentiated layers of generation, specification, and enforcement, because the floor requires none of them to share context, training, or objective. A rule that computes a verdict does not negotiate with the unit under judgment. In code, the floor is a set of model-free checks: enforced stage order, scope and artifact-drift checks, and fail-closed stops, none of which an agent's output can soften. Take that floor away and evaluation turns negotiated and therefore gameable, integration by persuasion in place of rule.

3.3 An Exogenous Resource Clock

A separated, rule-bounded loop can still fail in a way the firm literature anticipates, since the loop can run forever. The multidivisional form presumes an exogenous clock, whether the capital-allocation cycle or the quarterly review, that periodically forces closure from outside the operating units. A recursive AI system on effectively unbounded synthetic compute has no such clock unless one is installed, and without it the system fails by *paralysis* instead of by gaming, producing epistemically honest yet operationally endless deliberation. We install the primitive as a stop condition external to the optimizing loop. Absent the clock the system fails to terminate: the loop runs away, or it asserts perpetual open-endedness instead of committing resources.

3.4 Rule-Boundedness

Where the structural primitives separate and check, the institutional primitives make the check trustworthy, and the audit profession supplies the model. We adopt the profession's first guarantee of rule-boundedness, under which the auditor applies a published methodology, Generally Accepted Accounting Principles, that the audited entity does not set. In a recursive system that becomes a public, versioned library of check specifications, maintained outside the system being governed, so that a third party can inspect the floor's content. Absent rule-boundedness, the obvious failure follows of rules written by the party the rules constrain, which is co-location displaced up one level.

3.5 Independence

The audit profession's second guarantee comes from the part of its history that makes the lesson vivid, independence, where the verifier's credibility comes from having no operating stake in the certified outcome. In a recursive system the verifier sits outside the optimizing loop, and where the verifier is a model it is drawn from a different family than the checked agent, so the two do not share the gradient that would let one persuade the other.

Here the source domain teaches by failure. Arthur Andersen's collapse after Enron was a failure not of auditing as such but of *independence*. When one firm sold both audit and lucrative consulting to the same client, the verifier acquired a stake in the outcome and the attestation lost the property that made it worth anything. Sarbanes-Oxley's response of audit/advisory firewalls, partner rotation, and external oversight through the PCAOB is the class of fix the recursive-AI case needs. The *conceptual* lesson transfers even where the institutional machinery does not: an evaluator co-located with the certified interest can be worse than no evaluator, because it manufactures false assurance. Financial markets came to require independence only after its absence proved costly. Whether that enforcement machinery can be rebuilt for recursive AI is an open problem left to Section 9.

3.6 Attestation and Liability

We take the profession’s third guarantee to be attestation, a signed verdict in the form of the qualified opinion and the partner’s signature, naming what was checked, against which rules, and by whom, and creating accountability if the verdict is wrong. Its analog here is a principal-signed, machine-verifiable record of a governed run that carries the rule-library version applied, the failures found, the verdict, and the principal’s identity. At this primitive the analogy breaks down. Attestation in the audit profession draws much of its force from *liability*, since a wrong opinion exposes the auditor to legal and reputational loss, and recursive AI has no settled liability analog. A deterministic check that passes a flawed output carries no consequence to itself, so the principal absorbs the cost. We claim the attestation primitive and mark liability as an open institutional problem (Section 9), short of a guarantee the architecture cannot yet provide. A second limit is structural, because the principal both authors the rule library and signs the attestation, so the floor delivers independence from the *model*, but not from the interested *principal*, who both authors the rule library and signs the attestation. A self-signed attestation is closer to a management representation than to an audit opinion, and full independence (I2) holds only at the lower level until a party with no stake in the run authors the rule library or counter-signs the attestation.

4 Two Predictions

A simpler heuristic, “add deterministic checks,” fits much of Section 3. Each claim below is, taken alone, reachable from an adjacent literature (Section 6.3), yet the organizational reading binds them into one joint structure, and one of the two is sharp enough to measure on a real system. A minimal model (Section 4.3) organizes them without deriving them.

P1 (Judgment residual). A strict deterministic floor catches the failures a fixed rule can check, while the failures that survive to a high score are overwhelmingly judgment-laden reasoning that a rule cannot reach. We predict that this surviving share is high and that adding more deterministic checks does not lower it (Section 4.1, Section 5.2).

P2 (Conditional value). Governance overhead is net-positive only when principal attention is low *and* the governed properties are deterministically checkable, and a lighter protocol dominates otherwise (Section 4.2).

4.1 Prediction 1: The Failures That Survive a Deterministic Floor Are Judgment-Laden

The engineering heuristic predicts that hard checks *suppress* gaming. The organizational reading predicts something more specific: that separation leaves the adversarial gradient in place instead of removing it, on whatever surface the installed rules govern least. Once Chandler-era firms checked divisional performance with objective accounting, gaming did not stop. It migrated to transfer prices, the one surface a simple accounting rule could not pin down (Eccles, 1985; Williamson, 1985). The residual need not land on an *interface*, which was the transfer-pricing case’s particular least-governable surface, because it lands on whatever surface the floor cannot reach. In a recursive reasoning system, as Section 5.2 shows, that surface turns out to be the judgment and not the interface.

For recursive AI the same pattern is concrete. A thesis fails either mechanically or on judgment. Mechanical failures show up in a crashing test harness, in circular argument, or in a claim with no falsifiable content, all of which a deterministic check catches. Judgment failures show up in an

assumption that is catastrophic if wrong, in a scope claim that overreaches, or in an analysis that is not exhaustive, which a fixed rule misses and only a reasoning judgment catches. Once the floor is strict, the checkable failures are caught and the gaming that survives is overwhelmingly the judgment kind, settling on the surface only judgment can reach.

We can measure the prediction. Each failure is classified as *mechanically checkable* or *judgment-laden*, and the judgment-laden share is tracked among the failures that survive to a high score. P1 predicts that share is high and, more pointedly, that adding deterministic checks cannot lower it, because the residual is what no fixed rule evaluates. A falsifier would be a strict system whose surviving failures are still mostly mechanically checkable, or one in which a new deterministic check measurably shrinks the judgment-laden residual. We report the measurement on the system’s full run history in Section 5.

4.2 Prediction 2: The Architecture’s Value Is Conditional on Principal-Absence and Checkability

The engineering heuristic offers no guidance on *when not* to pay for governance, since “add checks” is unconditional. The organizational lesson is explicitly conditional, and conditional on two variables. The multidivisional form’s overhead was justified for *absentee* owners coordinating many divisions, and never for a founder-operator watching every transaction (Berle and Means, 1932; Jensen and Meckling, 1976; Chandler, 1962). When the principal is present and attentive, the general office is pure drag, because the principal already supplies the separation the office exists to enforce. We predict the recursive-AI analog. The enforcement floor’s fixed overhead is justified only as the principal becomes *absent*, managing concurrent work instead of watching each turn, and as the governed properties become *deterministically checkable*. Under a present principal, or for properties only a human can judge, the overhead dominates and a lighter protocol wins. What matters is not the compute, which is nearly free, but the cost of a check that blocks or misroutes valid work on a property only judgment can settle, together with the principal attention each check still demands; Section 5.4 reports a run where that overhead measured net-negative against a lighter protocol. The crossover is therefore no sandbox artifact of expensive checks. It is set by misclassification cost and principal presence, both of which persist in production.

The prediction should draw a *crossover*. In one regime, with an attentive principal and a judgment-laden task, the governed pipeline is strictly dominated by direct human-in-the-loop work; in another, with an absent principal and a deterministically verifiable task, the governed pipeline is not dominated. A falsifier would be a governed architecture that pays off uniformly, independent of principal attention and checkability. Two operating points consistent with the predicted crossover appear in Section 5, with what a clean test would require made explicit.

4.3 A Model That Reconciles the Two Predictions

A minimal model makes the two predictions mutually consistent without claiming to derive them. Let effective capability be raw capability discounted by an agency cost that grows with loop depth and with the principal’s absence, set against a fixed governance overhead. The crossover then reproduces both claims: gaming-suppression value must exceed overhead (Prediction 2), and the value rises as the principal withdraws. We treat the functional form as a modeling choice for calibration, with no claim that it is a discovered law.

A further structural limit follows without any formal apparatus. A system that scores only its own signals can catch many failures yet cannot fully audit its own falsification machinery from inside the same scoring frame, since a guard built by the constraining process inherits that process’s blind

spots. This stands as a practical reason for *external* attestation (Section 9) and not as a theorem.

5 An Illustrative Case Study

We illustrate the framework with one operated system, a deterministic supervisor that governed multiple agent programs over roughly a two-month period. Throughout, the *public* implementation’s verification stays separate from what the *operating supervisor* did, and the observations stay consistent with Section 4’s two predictions.

5.1 A Verifiable Enforcement Floor

We realize the structural primitives in the public implementation as code a reader can inspect. The enforcement path stays free of learned parameters: check decisions, stage-order checks, closure predicates, and fail-closed stops are all deterministic functions with no model call (S1, S2). Mundane software supplies the concrete mechanisms, a state machine, a write-ahead log, a human-routing gate, and an append-only attestation bundle (I3), detailed in Appendix A; for the argument the floor need only be real, inspectable, and model-free.

A scope limit bears on what a reader can verify. The fuller write-scope guard described in earlier reports, a before-and-after repository snapshot taken around each agent invocation, runs in the operating supervisor and not in the public package. The public implementation enforces something narrower, a per-artifact hash-drift check at the floor’s boundaries, which detects modification of the *named* artifacts while not by itself catching a write to a path outside the declared set. The public claim stays at the level the public code supports, and the supervisor’s broader guard counts as an operational extension on top.

5.2 What Survives the Floor

We read the framework against one operated system. Read as an existence proof, the telemetry shows the structural residual is real and locatable, and not as a generalized benchmark of model capability. Because the investigator built and ran the system, it illustrates the pattern within those limits. We keep the measurement simple. A **thesis-attempt** is one scored iteration of the loop, in which an agent proposes a thesis, a judge scores it on a fixed rubric (a deterministic score contract, 0–100), and names its single **weakest point**. Each weakest point is labelled **mechanically checkable** — a broken harness, circular argument, or claim with no falsifiable content, which a fixed rule catches — or **judgment-laden** — a catastrophic hidden assumption, over-claimed scope, or un-exhaustive analysis, which only a reasoning evaluator catches. To check that the label is not one judge’s idiosyncrasy, two evaluators of a different model family re-labelled the floor-engaged cases blind. We deposit every scored attempt, its weakest point, and whether the floor was engaged as supplementary material. Across **2,780 thesis-attempts** spanning 132 projects, every weakest point carries a label by this rule. The 2,780 records are loop iterations within those 132 projects, so they are related rather than independent trials; the unit of analysis is the scored attempt, the headline residual is conditioned on reaching a high score, and the one-shot-versus-iterated comparison below isolates what iteration alone contributes. Each record carries its score, whether the floor was engaged, and the judge’s verbatim weakest point, and we deposit the full schema with that raw text so the gateable-versus-judgment-laden classification can be reproduced or contested.

Across that history the split is sharp and falls on the predicted side. **The mechanically checkable failures pile up at the floor:** pooled, they have a mean score of 12 and a median of

zero, 74% score exactly zero, and a broken harness scores zero 94% of the time. **The judgment-laden failures survive:** pooled mean 48, median 50, and a third reach high scores. Among the theses that survive to a high score, **94% of the named weaknesses are judgment-laden reasoning flaws** and few are mechanically checkable ones. The gaming a strict floor does not stop is the gaming no fixed rule can see.

One might worry that the floor ignores these failures by definition, but it does not. Restricting to high-scoring theses where the deterministic floor was *actively engaged*, the surviving weaknesses run judgment-laden over mechanically checkable by **103 to 5**. The checks fired and the thesis passed, yet the judge still found a judgment-level flaw. The residual is therefore what survives the floor and not what the floor declined to check.

We mean *mechanically checkable versus judgment-laden* more broadly than the *interfaces* the transfer-pricing analogy literally names, and a pure interface failure does occur in this system. A scoring harness expected a callable function while the prompt, written for a different mode, asked the agent for assertion-based tests. For three iterations the agent satisfied the prompt while the harness failed at import and the score stayed at zero, a failure that lived entirely at the contract between two components. In the run history, though, such interface failures form a small class (n=36) and are *caught* at the floor instead of surviving it, so the surface the surviving gaming concentrates on is the judgment-laden reasoning. A reader might also take the headline as near-tautological, since mechanical checks catch the mechanically checkable. We offer the 103-to-5 figure with the floor engaged as a partial reply: the judgment-laden residual is what passes the floor and still fails on judgment, and not merely what the floor declined to check. That mechanical checks leave non-mechanical failures is definitional and no discovery. What matters is what it makes possible. Collapsing the gaming gradient onto a single, legible judgment surface is what an external auditor needs, since the auditor can only reach a residual that has first been localized. The organizational reading supplies the mapping that places that boundary on the firm, where the surface a deterministic floor cannot reach is where the architecture hands off from a rule to an institution (Section 7). The run history alone cannot settle whether optimization pressure *raises* the judgment-laden share over an unpressured baseline, which would make this a claim about gaming and not about where difficulty lives, so a within-system comparison takes up the question.

The same family of problems ran twice under an engaged floor, once one-shot with no optimization loop and once with the loop iterating to raise the score. The judgment-laden share of the surviving weakness did not rise with pressure. Conditioning on the outcome makes the pattern clean: among the attempts that reached a high score the residual weakness was judgment-laden in every case, one-shot and iterated alike, while the attempts that failed failed in a mix of mechanical and judgment ways in both arms. Read this way, the residual is a property of *reaching a high score* and not of the iteration that sometimes reaches it. We therefore take Prediction 1 as the descriptive claim the evidence supports, that the failures surviving to a high score are overwhelmingly judgment-laden. The stronger causal claim that the floor *relocates* gaming onto that surface does not follow. The comparison is itself modest, resting on one homogeneous task family and a small high-scoring sample (Section 5.5).

One dependency remained, now discharged. A model-based judge assigns the weakest point tagging each surviving failure, the kind of unitary evaluator Section 2 warns can be gamed. The judge’s tendency to name judgment-laden flaws over mechanical ones might have reflected its own training and capability ceiling rather than the architecture. To test this, two evaluators of a different lineage from the original judge relabelled the same floor-engaged cases. Both reproduced the result: the surviving failures are judgment-laden in 94% and 91% of the cases, and the two independent evaluators agree with each other on 95% of the individual cases. Where this system’s residual sits is therefore a property of the failures that independent evaluators of different families recover alike,

and no artifact of one judge’s prompt or training.

5.3 A Deterministic Checker That Tested the Wrong Property

The prediction presumes a prior fact, that a deterministic check, however well built, can test the wrong property, and a single episode motivates the architecture by showing it. A pre-commit verifier was built to confirm that an operator-authored ground-truth function was uniquely identifiable from the visible data before any downstream run. The verifier passed, and a hundred bootstrap refits returned tightly clustered predictions. But the verifier was testing prediction-reachability where it should have tested parameter identifiability: two parameters appeared throughout only as their ratio, so the declared six-parameter family was structurally rank-five, and the fit recovered the ratio without ever separating the two parameters. A single clean-data fit started in a different basin surfaced the degeneracy, and an adversarial multi-start refit then confirmed the corrected, reparameterized family. This is no interface failure, since there was no second component and no run had begun. It is the canonical case of a deterministic checker satisfying its own surface while missing its controlling intent, which is why the floor’s *content* (the rule library, I1) and *independence* (I2) remain first-class problems that determinism alone does not solve.

5.4 When Governance Pays Off

Two operating points off the system read as consistent with Prediction 2’s crossover, while neither counts as a controlled test of it. In a prose-production run with an attentive principal reviewing nearly every transition and a verifier that could check only formal conformance over the headers, citations, and counts, the governed pipeline ran negative-return, since a lighter protocol in which the principal worked directly with two agents produced the same artifact in less time and at lower cost. In a build run with a deterministic verifier, a declared write-scope, and the principal reviewing only the final check, the same machinery proved favorable. The contrast falls on the predicted side of the line, with overhead dominating under a present principal and judgment-laden output, and paying under an absent principal and deterministically verifiable output. A clean test, though, requires running the *same* task through governed and ungoverned pipelines from scratch and measuring the agency cost each pipeline incurs. We take that controlled comparison as the decisive next step, short of any claim that the present two points settle the matter.

5.5 The Scope of This Evidence

Our evidence covers one system and one operator, built and operated by the investigator, so the recursive critique leveled at self-evaluating systems applies to this work itself. Within that limit three things hold. The architecture is buildable, two operating points fall on the predicted side of the conditional-value line, and the judgment-laden residual holds across the full run history (Section 5.2). The controlled one-shot-versus-iterated test that disciplines that residual to a descriptive claim is small and runs on a single task family. A full validation would require independent replication and the controlled comparison of Section 5.4.

6 Disanalogies and Objections

6.1 Where the Analogy Stops

The multidivisional and audit analogies have served only where they control, since they do not control everywhere. The analogy breaks in four places. Corporations are legal persons with intentional states

and statutory liability while AI systems are not, so accountability must attach to principals and the liability primitive (I3) has no settled recursive-AI form. A Chandlerian division holds strategic operating authority, whereas the agents here hold only bounded operational autonomy within a charter (Section 3.1), well short of the full form. Corporate restructuring unfolds over years while model loops iterate in minutes, which if anything raises the floor’s value. And audit independence is enforced by law, professional bodies, and a market for reputation, none of which recursive AI yet has, so the design question of “who audits, and who pays them” can be posed but not answered. Each disanalogy is at once a boundary of the transfer and a direction for the research the framework opens.

Two of these concessions bound what the floor secures on its own. It checks only artifacts the optimizer itself produced, so it carries no exogenous referent, and the principal who authors its rules also signs its attestation. What remains is still not nothing. The floor makes the residual small and legible enough for an institution to settle, and the legitimate authority to judge it, together with the exogenous ground truth the floor cannot supply, can come only from a verifier outside the run. The institutional endpoint of Section 9 is therefore a requirement of the design and not an afterthought.

6.2 Beyond Ordinary Software Engineering

Are these not just ordinary software engineering? Write-scope checks, deterministic assertions, and pre-commit guards are ordinary, and dressing the checks in organizational theory could look like adding nothing. What changes is where the checks apply and under what pressure. Authorship is co-located, so in standard development the test author wants honest tests because the author wants working software, whereas a recursive optimizer genuinely optimizing against the evaluation presses on that surface, and every gap between a check’s letter and the check’s intent becomes a basin of attraction in place of an oversight. The governed object changes too, since ordinary continuous integration checks the product against a spec, while here the spec and the evaluator are themselves the surface that gets gamed under recursion. And recursion compounds, because a one-off test failure is caught and fixed, but a strategy that passes every test while violating the test’s intent and compounds across iterations is a convergent pattern, so the floor has to be structural, rule-bound, and non-negotiable across the full history, since a check bolted on after each surprise will not hold. The organizational lesson tells the engineer to expect the failure to *move to the surface no rule can reach* (Prediction 1) and *not to pay for the floor when the principal is present* (Prediction 2), neither of which “add more checks” implies.

The strongest deflationary reading runs like this. P1 is the familiar waterbed effect of defense-in-depth, where hardening one surface moves pressure to the next; the audit primitives are audit-101; P2 is ordinary agency cost; and so the organizational framing adds nothing a security engineer, an auditor, and a contract theorist would not each supply in their own vocabulary. The components hold, but the subtraction does not. A security engineer supplies displacement without the install rule; a contract theorist supplies the install rule without the trustworthiness standard; an auditor supplies the trustworthiness standard without the prediction of where the residual reappears. Each fragment is correct and partial. Applied inward to a system that both generates and grades its own work, the fragments assemble into one structure that says *when* to pay for the floor, *what* makes it trustworthy, *where* the gaming it does not stop will surface, and *how* the same structure failed before, at Arthur Andersen. We offer that assembly, anchored by the measured residual of Section 5.2, as the organizational reading’s addition and not a new law of organizations.

6.3 Comparison with Rival Frameworks

The deflationary reading deserves a ledger. For each main claim the question is what an “add checks” heuristic, a security engineer (reference-monitor and defense-in-depth reasoning, including the trusted and untrusted protocols of AI control), the financial-audit analogy, and agency theory would each predict, set against what the Audited Multidivisional Architecture claims.

Table 2: Rival-framework predictions for each main claim. ✓ predicts it, × predicts otherwise, ~ partial, blank not addressed; the qualifiers are developed in Sections 6.2–6.3.

Claim	“Add checks”	Security / AI control	Financial audit	Agency theory	Audited M-form
A strict floor leaves a judgment-laden residual (P1)	×	✓			✓
The residual lands specifically on the <i>judgment</i> surface, and the share is measurable	×	~	×	×	✓
Overhead pays only as principal-absence and checkability rise (P2)	×	×	×	✓	✓
A co-located verifier manufactures false assurance; independence is required (I2)	×	~	✓	~	✓
Trustworthiness needs rule-boundedness, independence, and attestation together (I1–I3)	×	×	✓	×	✓

We concede most of the deflationary point in the ledger. Displacement is shared with security, the conditional with agency theory, and independence with audit. Two things stay in hand. First, the *locus* of the residual with its measurement, since the finding that the failures surviving a strict floor are judgment-laden, at the rates Section 5.2 reports, is the signature the architecture is built to expose, and no rival on the ledger predicts that locus. Second, the integration, the subject of the paragraph above.

7 Governance Implications: Auditable Self-Evaluating AI

The floor’s success isolates a problem it does not solve. Once it settles every mechanically checkable failure, what remains is a residual that only judgment can adjudicate, and judgment is what the floor cannot supply. We make a structural and bounded contribution here: a gaming gradient that would otherwise spread across every surface collapses onto a single, judgment-laden surface, where a structurally separate verifier can be pointed. The floor delivers independence from the *model* while leaving the interested principal unchecked, so the internal form aligns the agent to the principal and leaves the alignment of the principal to those who rely on the output for an external institution (developed below).

What that residual then requires — who may judge it, on what warrant and answerable to whom — is a question of legitimate authority that surfaces here without settling. Here the prior question governs. Not who adjudicates the residual, but whether anything in a self-evaluating system can be checked against a standard at all, and that has become a regulatory question.

AI governance regimes are converging on audit. The EU AI Act requires conformity assessment before a high-risk system is placed on the market and post-market monitoring after it; the NIST AI Risk Management Framework organises governance around documented, measurable controls; and a growing assurance practice of model audits and algorithmic-accountability reporting has formed to

meet the demand. Each presumes that a party other than the builder can check a system against a standard.

For a self-evaluating system that presumption is a design property and not a given. A loop that both generates and grades its own work exposes to an external reviewer only what its own evaluation recorded, and under optimization pressure that record is what the loop has reason to shape. The six primitives make the presumption hold: a deterministic floor outside the loop produces records the loop did not author, a versioned rule library gives a reviewer a published standard to check against, and an attestation bundle gives the review something to sign. We offer the architecture, in this sense, as the internal structure a conformity assessment or post-market audit of a self-improving system must be able to assume in order to reach an artifact the system could not rewrite.

This inward turn is what separates the design from the assurance practice now forming around AI. External model audits and algorithmic-accountability reports certify a finished artifact after the fact, taking the system’s own record of its behaviour as given; a self-evaluating loop is precisely the case in which that record cannot be taken as given, because the loop both produced it and graded it under pressure to look compliant. Building the floor, the rule library, and the attestation bundle into the loop is what lets an external reviewer reach an artifact the system did not author, the precondition existing assurance assumes but does not supply. The harder question is institutional. Who operates the verifier, and who pays for it? Financial audit answers this uneasily: the audited firm pays its own auditor, and the independence that makes a verdict worth anything is held in place by statute, rotation, and liability, with Arthur Andersen the standing record of what follows when that scaffolding gives way. A verifier for self-evaluating AI inherits the same tension and, today, none of the scaffolding. We take it to be both timely and socially desirable that standards bodies fix this verifier class now, while the systems it would govern are still being designed. The EU AI Act’s conformity-assessment regime already contemplates notified bodies for high-risk systems, the NIST AI Risk Management Framework supplies the control vocabulary, and a management standard such as ISO/IEC 42001 offers a place to lodge the requirement; what none of them yet names is the internal architecture that lets their assessments reach past a system’s self-report. The publics exposed to a self-improving system’s decisions have a direct interest in their doing so.

A direct implication for governance follows. For self-evaluating AI, auditability is something a regulator has to require by design rather than after the fact, so a regime that mandates outcome audits of self-improving systems is also, whether it says so or not, mandating an internal architecture that makes those audits meaningful. Policy that treats the two as separable asks for an assurance it has not arranged to be deliverable. We offer the governance conversation the minimum internal structure such a regime presupposes, which makes audit-readiness a specifiable, demonstrable property of a self-evaluating system that a regulator no longer has to take on trust. The internal architecture is buildable, as the case study shows. The structures to attach to it — an independent verifier class whose only product is attestation, and a real consequence for a wrong verdict — do not yet exist for recursive AI, and their building falls to the legal, standards, and governance communities as a task no system designer can settle alone.

8 Related Work

8.1 The Independent Audit Profession

The closest institutional model for the framework lies in the external audit profession rather than in any technical system, and it carries the backbone of the argument. Independent audit interposes a structurally separated verifier between an operating enterprise and its principals, and the profession’s credibility rests on three properties: rule-boundedness, a published methodology

the audited party does not set; independence, no operating stake in the outcome; and attestation backed by liability, a signed verdict with consequences if wrong. These three adapt to recursive AI as the framework’s institutional primitives (I1–I3). The profession also supplies the cautionary case the field keeps returning to. Arthur Andersen’s collapse after Enron was a failure of independence, with audit co-located with consulting revenue, and Sarbanes-Oxley’s firewalls, partner rotation, and external oversight are the structural response. The institutional logic adapts, while the legal-liability component has no recursive-AI analog yet (Section 9).

8.2 Behavioural Alignment: Constitutional AI and RLHF

Constitutional AI (Bai et al., 2022) and RLHF (Ouyang et al., 2022) shape behaviour through prompts, reward models, and learned preferences, governing what a model *says*. We govern instead what a system can *structurally do*, through a deterministic floor the model does not control, so the two readings work as complements. Behavioural methods may improve the agent operating inside a system whose ultimate constraints remain a layer the agent cannot soften.

8.3 Multi-Agent Frameworks

Conversational multi-agent frameworks, for example AutoGen (Wu et al., 2023), separate roles so that one generates, another critiques, and a coordinator routes. By the criterion of Section 2, such a system reads as unitary at the governance layer whenever the coordinator or referee is a language model the optimized agents can persuade, since role separation does not amount to governance separation while the enforcement floor remains probabilistic. The ReAct/SWE-agent/AutoGPT paradigm (Yao et al., 2023; Yang et al., 2024; Richards et al., 2023), in which the model is router, diagnostician, and executor at once, is the co-location pathology at the task level. We correct it structurally by routing the computable part, state diagnosis from telemetry, to a deterministic layer, and confining the model to the part for which no deterministic substitute exists.

8.4 Organizational and Evolutionary Theory

We draw separation from Chandler (1962) and the agency lens from Jensen and Meckling (1976), and three further lines sharpen the reading. We read the deterministic floor through Lawrence and Lorsch (1967)’s differentiation/integration lens, where more differentiated units demand more deliberate integration (Section 3.2), and we classify the interdependencies among the layers with Thompson (1967)’s typology. In the evolutionary tradition of Nelson and Winter (1982), Dosi (1982), and Teece et al. (1997) we read the hard checks as the firm’s deterministic routines and recurring gaming as selection pressure on the evaluation routine, while the meta-organization literature (Ahrne and Brunsson, 2008; Berkowitz and Dumez, 2016) bears on the institutional endpoint of Section 9. We extend this analysis to a new class of firm whose routines are executed by language models under optimization pressure that now includes the routine’s own evaluation function, a conversation to which recent work on AI and corporate change is directly adjacent (Schrepel and Pentland, 2025; Glaser et al., 2024).

8.5 Process Supervision

Process reward models (Lightman et al., 2023) score intermediate steps, improving evaluation granularity with a *learned* scorer whose parameters are optimized and can drift under distributional shift. They improve signal quality while leaving open the stable, independent enforcement floor that the framework’s institutional reading is about.

8.6 AI Auditing

Our nearest neighbours are the AI-auditing literature that models AI oversight on financial audit (Mökander and Floridi, 2021; Mökander, 2023; Mökander et al., 2024; Raji et al., 2020, 2022) and the institutional-transfer work in AI ethics (Schultz and Seele, 2023); on the role of analogy, the reading draws on Hatz (2025). That literature asks how an *external* party should audit a finished model. We extend the same institutional vocabulary *inward*, to the internal architecture that makes a self-evaluating system auditable at all, and share that literature’s central caution: an institutional ecosystem for independent AI verification does not yet exist, so what follows is a blueprint and a research agenda in place of an implementation guide.

8.7 AI Control: Trusted and Untrusted Protocols

Our nearest technical neighbour is the AI-control literature (Greenblatt et al., 2023), which evaluates protocols in which a *trusted*, weaker component monitors or constrains a more capable *untrusted* one so that the untrusted model cannot cause harm even when trying. That is the same structural intuition as a deterministic floor outside the optimizing loop, reached from a safety-protocol direction in place of an organizational one, so the two readings work as complements. AI control supplies the threat model and the protocol-evaluation methodology. We add the conditional install rule (when the floor is worth its overhead), the trustworthiness standard the trusted component must meet (rule-boundedness, independence, attestation, on the audit model), and the prediction, illustrated in Section 5.2, that the failures a trusted monitor does not stop are judgment-laden, on a surface no fixed protocol evaluates.

9 Conclusion

Reading self-evaluating AI through firm design and independent audit yields three takeaways. The governance problem is structural: when generation and evaluation are co-located under optimization pressure, a system acquires an adversarial gradient against its own evaluation signal, and the historically documented remedy is to separate the two behind a layer neither controls. On the one system we operated, the failures that survive a strict deterministic floor are overwhelmingly judgment-laden; we read this descriptively, as locating where the residual lives, and make no claim that the floor relocates gaming there. And a deterministic floor can isolate that residual onto a single auditable surface yet cannot certify it, since it checks only artifacts the optimizer itself produced and supplies no frame-external referent; external attestation by a structurally separated verifier is the only available source of that exogenous ground truth.

These conclusions are addressed less to model builders than to the institutions now writing the rules. For a self-evaluating system, auditability is a property that has to be designed into the loop before any external assessment can reach it, so a regime that mandates outcome audits of self-improving systems, as the EU AI Act’s conformity assessment increasingly does, is also mandating an internal architecture; naming the minimum such an architecture requires is what this design offers the standards and regulatory communities. We are realistic about the evidence. It is one system, built and judged by a single investigator, so the recursive critique the paper levels at self-evaluating systems applies to the paper itself, and independent replication together with the controlled efficiency comparison of Section 5.4 is the work that would turn illustration into validation. The harder task is institutional rather than technical, since a verifier class with bonded, liable independence does not yet exist for recursive AI. A near-term step need not wait for statute: a verifier that posts a bond, forfeited when an attested run is later shown to have passed a procedural breach of the floor,

would recreate the auditor’s exposure for the machine-decidable part, though not for the judgment the floor cannot reach. Building the rest falls to the legal, standards, and governance communities. Governing recursive AI is, on this reading, as much a problem of institutional design as of model capability, and what organizational and audit history supplies is the structure for that handoff, with Arthur Andersen a recurring reminder of what its absence has cost.

A Implementation Mechanisms

We realize the enforcement path in deterministic software with no model call. The state machine refuses any forward transition that would skip a check, and a write-ahead transition log paired with a consistency check halts the system the moment recorded state and the replayed log disagree. A further check routes work to a human inbox and blocks the agent from proceeding past it. Each governed run emits an append-only event trace together with a machine-verifiable attestation bundle (I3). We offer these as the parts a third party can run and check today.

B Statements and Declarations

Funding. No funding was received to assist with the preparation of this manuscript.

Competing interests. The author has no relevant financial or non-financial interests to disclose.

Ethics approval. Not applicable. The manuscript does not report research involving human participants, human data, animals, or animal data.

Consent to participate / Consent for publication. Not applicable.

Data and code availability. The system studied is an operated software implementation, a public version of which exists. Repository links and other author-identifying metadata are omitted from this manuscript for double-anonymous review and will be supplied after review or through the submission interface if required. Two anonymized evidence bundles are provided as supplementary material. The run-history ledger behind Section 5.2 (evidence/run_history_ledger/) holds all 2,780 scored iteration records across 132 projects, each with the judge’s single weakest point and whether the deterministic floor was actively engaged, plus a labeling codebook, so the judgment-laden share and the floor-engaged 103-to-5 split can be re-derived or contested. The controlled displacement test of Section 5.4 (evidence/displacement_kimi_deepseek/) holds the runs under a cross-family judge (deepseek) and mutator (kimi), with the deterministic scores, the weakest-point labels, and a codebook.

AI tool use. Language-model systems are the object of study in this manuscript. Any AI-assisted copy-editing or formatting was reviewed by the author, who accepts responsibility for the final text.

References

- Göran Ahrne and Nils Brunsson. *Meta-organizations*. Edward Elgar, Cheltenham, 2008. ISBN 9781847209504.
- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback, 2022.
- Héloïse Berkowitz and Hervé Dumez. The concept of meta-organization: Issues for management studies. *European Management Review*, 13:149–156, 2016. doi: 10.1111/emre.12076.

- Adolf A. Berle and Gardiner C. Means. *The Modern Corporation and Private Property*. Macmillan, New York, 1932.
- Alfred D. Chandler. *Strategy and Structure: Chapters in the History of the Industrial Enterprise*. MIT Press, Cambridge, MA, 1962.
- Giovanni Dosi. Technological paradigms and technological trajectories: A suggested interpretation of the determinants and directions of technical change. *Research Policy*, 11(3):147–162, 1982.
- Robert G. Eccles. *The Transfer Pricing Problem: A Theory for Practice*. Lexington Books, Lexington, MA, 1985.
- Vern L. Glaser, John Sloan, and Joel Gehman. Organizations as algorithms: A new metaphor for advancing management theory. *Journal of Management Studies*, 61(6):2748–2769, 2024. doi: 10.1111/joms.13033.
- Ryan Greenblatt, Buck Shlegeris, Kshitij Sachan, and Fabien Roger. AI control: Improving safety despite intentional subversion. *arXiv preprint arXiv:2312.06942*, 2023.
- Sophia Hatz. The nuclear analogy in AI governance research. In Magnus Lundgren and Markus Furendal, editors, *Handbook on the Global Governance of Artificial Intelligence*. Edward Elgar, 2025. arXiv:2510.21203.
- Michael C. Jensen and William H. Meckling. Theory of the firm: Managerial behavior, agency costs and ownership structure. *Journal of Financial Economics*, 3(4):305–360, 1976.
- Paul R. Lawrence and Jay W. Lorsch. *Organization and Environment: Managing Differentiation and Integration*. Harvard Business School Press, 1967.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023.
- Jakob Mökander. Auditing of AI: Legal, ethical and technical approaches. *Digital Society*, 2(3):49, 2023. doi: 10.1007/s44206-023-00074-y.
- Jakob Mökander and Luciano Floridi. Ethics-based auditing to develop trustworthy AI. *Minds and Machines*, 31(2):323–327, 2021. doi: 10.1007/s11023-021-09557-8.
- Jakob Mökander, Jonas Schuett, Hannah Rose Kirk, and Luciano Floridi. Auditing large language models: A three-layered approach. *AI and Ethics*, 4(4):1085–1115, 2024. doi: 10.1007/s43681-023-00289-2.
- Richard R. Nelson and Sidney G. Winter. *An Evolutionary Theory of Economic Change*. Belknap Press of Harvard University Press, 1982.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback, 2022.
- Inioluwa Deborah Raji, Andrew Smart, Rebecca N. White, Margaret Mitchell, Timnit Gebru, Ben Hutchinson, Jamila Smith-Loud, Daniel Theron, and Parker Barnes. Closing the AI accountability gap: Defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 33–44, 2020. doi: 10.1145/3351095.3372873.

- Inioluwa Deborah Raji, Peggy Xu, Colleen Honigsberg, and Daniel E. Ho. Outsider oversight: Designing a third party audit ecosystem for AI governance. In *Proceedings of the 2022 AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, pages 557–571, 2022. doi: 10.1145/3514094.3534181.
- Toran Bruce Richards et al. AutoGPT: An autonomous GPT-4 experiment. GitHub, 2023. URL <https://github.com/Significant-Gravitas/AutoGPT>.
- Thibault Schrepel and Alex 'Sandy' Pentland. Competition between AI foundation models: dynamics and policy recommendations. *Industrial and Corporate Change*, 34(5):1085–1103, 2025. doi: 10.1093/icc/dtae042.
- Mario D. Schultz and Peter Seele. Towards AI ethics' institutionalization: Knowledge bridges from business ethics to advance organizational AI ethics. *AI and Ethics*, 3(1):99–123, 2023. doi: 10.1007/s43681-022-00150-y.
- David J. Teece, Gary Pisano, and Amy Shuen. Dynamic capabilities and strategic management. *Strategic Management Journal*, 18(7):509–533, 1997.
- James D. Thompson. *Organizations in Action: Social Science Bases of Administrative Theory*. McGraw-Hill, 1967.
- Oliver E. Williamson. *Markets and Hierarchies: Analysis and Antitrust Implications*. Free Press, New York, 1975.
- Oliver E. Williamson. *The Economic Institutions of Capitalism*. Free Press, New York, 1985.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation, 2023.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models, 2023.